Graph node & edge types

The overall graph structure and edge storage model are defined in the general database schema. This document completes that design by defining the supported node and edge types, their meanings, and the canonical direction of each edge relationship.

`11` node types | `6` edge types | `8` implementation rules | controlled vocabularies

01

Graph node & edge types

Controlled vocabularies

The graph should use documented, controlled vocabularies for both `nodes.node_type` and `edges.edge_type`.

- `node_type` identifies what kind of FAR source object a node represents.
- `edge_type` identifies the meaning of a non-hierarchical relationship between two nodes.
- FAR hierarchy should remain authoritative in `nodes.parent_id`.
- Edge direction is represented by the existing ordered fields `from_node_id` and `to_node_id`; a separate direction column is not needed.
- Each relationship should be stored once in its canonical direction. Reverse relationships should be derived by querying the same edge from the opposite endpoint rather than storing duplicate inverse edges.

For example:

```
from_node_id: FAR_1_301_A_2
to_node_id: FAR_1_105
edge_type: references
```

means:

```
FAR 1.301(a)(2)
→ references
→ FAR 1.105
```

The reverse relationship, *“FAR 1.105 is referenced by FAR 1.301(a)(2),”* is derived by querying incoming `references` edges where `to_node_id = FAR_1_105`.

02

Node types

Structural level and regulatory function stay separate

For example, a Part 52 clause may be stored as:

```
node_type: subsection
instrument: clause
citation: 52.204-24
```

rather than using `clause` as the node type. This preserves the distinction between:

- where the source item appears in the FAR hierarchy; and
- what regulatory function it performs.

NODE TYPE	MEANING	INITIAL STATUS
regulation	Root node for the FAR corpus	Initial
subchapter	FAR subchapter grouping	Initial
part	FAR Part, such as Part 9	Initial
subpart	FAR Subpart, such as Subpart 9.4	Initial
section	FAR Section, such as 9.403 or 1.102	Initial
subsection	Numbered unit below a section, such as 1.102-2 or 52.204-24	Initial
paragraph	Addressable paragraph or nested FAR subdivision, such as 1.102-2(a) or 1.102-2(a)(4)	Initial
definition	Individually addressable defined term and definition	Add with scoped-definition retrieval
table	Table retained as its own source and retrieval item	Add with table-aware ingestion
image	Image, diagram, or formula retained as its own source item	Add with image-aware ingestion
external_resource	Outside authority or source represented as a graph node	Optional future use

Nested subdivisions. For nested FAR subdivisions, use one generic `paragraph` node type rather than creating a new node type for every nesting level.

Example:

```
citation: 1.102-2(a)(4)
node_type: paragraph
paragraph_path: (a)(4)
paragraph_depth: 2
```

```
parent_id: FAR_1_102_2_A
```

This supports deeper FAR subdivision patterns without requiring separate types such as `subparagraph`, `sub-subparagraph`, and additional nested variants.

Clause and provision should remain values in the existing `instrument` field:

```
instrument: clause
instrument: provision
```

Other regulatory functions may be added to the `instrument` vocabulary as the corpus requires.

03

Edge types

A small, reliable vocabulary

The initial edge vocabulary should remain small and should include relationships that can be identified reliably and used by retrieval.

EDGE TYPE	CANONICAL STORED DIRECTION	MEANING	INITIAL STATUS
references	Referencing FAR node → referenced FAR node	General internal FAR cross-reference	Initial
prescribed_by	Clause or provision → prescribing FAR section	Identifies the FAR section that prescribes use of a clause or provision	Initial
defines	Defining FAR section → definition node	Identifies the section that establishes a definition	Add with scoped-definition retrieval
applies_within	Definition node → applicable structural scope	Identifies the Part, Subpart, Section, clause, or other scope in which a definition applies	Add with scoped-definition retrieval
more_specific_than	Narrower definition → broader definition	Supports choosing the most specific applicable definition when the same term has multiple definitions	Add with scoped-definition retrieval

EDGE TYPE	CANONICAL STORED DIRECTION	MEANING	INITIAL STATUS
`deviates_from`	Deviation or supplement node → affected FAR node	Connects supplemental or deviation content to the underlying FAR requirement	`Future multi-regulation support`

Not yet included. `incorporates_by_reference` is not included in the current vocabulary. It is a more specific legal relationship than an ordinary cross-reference, but the current FAR DITA graph does not yet have a reliable rule or retrieval requirement for distinguishing it from `references`.

External legal references should remain in the separate `external_references` structure for the initial implementation rather than being stored as internal FAR graph edges.

04

Hierarchy relationships

Parent–child stays in the tree

Parent-child hierarchy should remain authoritative in `nodes.parent_id`.

The edge table should not duplicate hierarchy with stored edge types such as `parent_of`, `child_of`, `contains`, or `part_of`.

For example:

```
FAR
→ Part 9
→ Subpart 9.4
→ Section 9.403
```

is represented through each node's `parent_id`.

Reverse hierarchy traversal is derived by querying nodes whose `parent_id` equals the current node.

> This avoids maintaining the same structural relationship in both **nodes.parent_id** and **edges**.

05

Inverse relationships

Derived, never duplicated

Inverse relationship names may be useful in APIs, graph-query results, or user-facing explanations, but they should not require duplicate edge records.

STORED EDGE TYPE	STORED DIRECTION	DERIVED INVERSE LABEL
`references`	Referencing node → referenced node	`referenced_by`
`prescribed_by`	Clause/provision → prescribing section	`prescribes`
`defines`	Defining section → definition	`defined_by`
`applies_within`	Definition → applicable scope	`has_applicable_definition`
`more_specific_than`	Narrower definition → broader definition	`has_more_specific_definition`
`deviates_from`	Deviation/supplement → affected FAR node	`has_deviation`

For example, only this edge is stored:

```
FAR 1.301(a)(2)
→ references
→ FAR 1.105
```

The graph API may expose the reverse result as:

```
FAR 1.105
← referenced_by
← FAR 1.301(a)(2)
```

but no second database edge is required.

06

Implementation rules

Eight rules for a stable vocabulary

1. `node_type` and `edge_type` should use controlled vocabularies rather than unrestricted free-text values.
2. The canonical direction of every edge type should be documented and used consistently by all parsers and ingestion processes.
3. Edge direction should be determined by:

   ```
   from_node_id
   → edge_type
   → to_node_id
   ```

   No separate `direction` column is required.
4. Each relationship should be stored once. Reverse traversal should use indexes on both endpoints:

   ```
   edges(from_node_id, edge_type)
   edges(to_node_id, edge_type)
   ```

- FAR hierarchy should be stored once through `nodes.parent_id`, not duplicated as edge rows.
- `node_type` should represent structural or source type. Regulatory functions such as `clause` and `provision` should remain in the separate `instrument` field.
- New node or edge vocabulary values should be added only when there is:
  - a reliable extraction or creation rule;
  - a defined graph or retrieval use case; and
  - documented direction and semantics.
- The vocabulary may be extended without changing the core node/edge table structure.